

Obligatorisk oppgave 6: Synkronisering

Andreas Isaksen

October 2, 2024

Oppgave 1

Følgende C-program bruker to tråder som begge ”teller opp” og ”teller ned” samme delte variabel `count`. Første tråd legger til 1 på variabelen 100 millioner ganger, andre tråd trekker 1 fra variabelen 100 millioner ganger:

```
#include <stdio.h>
#include <pthread.h>

int count = 0;

void *increase()
{
    int i;
    for(i = 0; i < 1e8; ++i)
        count++;
}

void *decrease()
{
    int i;
    for(i = 0; i < 1e8; ++i)
        count--;
}

int main()
{
    pthread_t thread1, thread2;

    pthread_create(&thread1, NULL, increase, NULL);
    pthread_create(&thread2, NULL, decrease, NULL);

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    printf("count = %d\n", count);
}
```

a) Kjør dette programmet selv, flere ganger. Du vil se at verdien av `count` er forskjellig hver gang. Selv om den ene tråden gjør ”++” like mange ganger som den andre gjør ”--”, blir aldri variabelen lik 0! Gi en forklaring på hvorfor dette skjer.

Svar:

Grunnen til at vi får uforutsigbare slutninger på denne prosessen er på grunn av *race conditions*. Her har vi to tråder som starter på samme tid i to forskjellige funksjoner, og begge tråder jobber mot den samme globale variabelen uten

synkronisering.

Dette medfører at begge tråder prøver å lese av verdien til *count* og subtrahere/addere til variabelen på samme tid siden de jobber mot samme lokasjon i minnet.

Siden operatorene *count++* og *count--* ikke er atomære, så blir denne operasjonen brutt ned til tre steg på maskinelt nivå for hver iterasjon:

1. Leser nåværende verdi for *count*.
2. Adderer/subtraherer verdien.
3. Skriver den nye verdien tilbake til *count*.

I praksis kan dette lede til at begge tråder leser av feil verdi på variabelen, samt at når de prøver å skrive en ny verdi til *count* på samme tid, så vil den ene tråden overskrive output fra den andre.

b) Bruk en mutex til å "reparere" programmet slik at *count* alltid blir lik 0. Du skal ikke skrive om noe av den eksisterende koden, men bare legge til bruk av en mutex.

```
#include <stdio.h>
#include <pthread.h>

//Legger til mutex som en variabel
pthread_mutex_t mutex;

int count = 0;

void *increase()
{
    //Laaser variabelen count frem til iterasjonen er
    //ferdig.
    pthread_mutex_lock(&mutex);
    int i;
    for(i = 0; i < 1e8; ++i)
        count++;
    //Laaser opp variabelen etter at iterasjon er
    //ferdig.
    pthread_mutex_unlock(&mutex);
}

void *decrease()
{
    //Laaser variabelen count frem til iterasjonen er
    //ferdig.
    pthread_mutex_lock(&mutex);
```

```

int i;
for(i = 0; i < 1e8; ++i)
    count--;
    //Laaser opp variabelen etter at iterasjon er
    ferdig.
    pthread_mutex_unlock(&mutex);
}

int main()
{
    pthread_t thread1, thread2;

    //Initialiserer mutex.
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&thread1, NULL, increase, NULL);
    pthread_create(&thread2, NULL, decrease, NULL);

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    // Terminerer mutex naar traader er ferdig kjort.
    pthread_mutex_destroy(&mutex);

    printf("count = %d\n", count);
}

```

Listing 1: oppgave1.c

Oppgave 2

Programmet nedenfor bruker to mutexer:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t mutex1, mutex2;

void* print1()
{
    pthread_mutex_lock(&mutex1);
    usleep(100000);
    pthread_mutex_lock(&mutex2);

    printf("Her er funksjonen \"print_1\"!\n");

    pthread_mutex_unlock(&mutex2);
    pthread_mutex_unlock(&mutex1);
}

void* print2()
{
    pthread_mutex_lock(&mutex2);
    usleep(100000);
    pthread_mutex_lock(&mutex1);

    printf("Her er funksjonen \"print_2\"!\n");

    pthread_mutex_unlock(&mutex1);
    pthread_mutex_unlock(&mutex2);
}

int main()
{
    pthread_t t1, t2;

    pthread_mutex_init(&mutex1, NULL);
    pthread_mutex_init(&mutex2, NULL);

    pthread_create(&t1, NULL, print1, NULL);
    pthread_create(&t2, NULL, print2, NULL);

    while (1)
    {
```

```

        putchar( ' . ' );
        fflush( stdout );
        usleep( 100000 );
    }
}

```

Hovedprogrammet `main()` initialiserer her de to mutexene, og oppretter så to tråder. Første tråd starter med å kjøre funksjonen `print1()`, andre tråd kjører `print2()`. Deretter går `main()` inn i en evig løkke som i hver iterasjon skriver ut et punktum og deretter ”sover” i et tidels sekund.

Kjør dette programmet selv. Du vil da se at ingen av de to trådene skriver ut noen meldinger, selv om begge kaller `printf()`!

Spørsmål: Gi en forklaring på hvorfor ingen av trådene får eksekvere `printf()`.

Svar:

Det som forekommer i denne kjøringen kalles en *deadlock*.

Problemet her er at *Tråd1* låser *mutex1* og venter på å låse *mutex2*.

På samme tid har *Tråd2* låst *mutex2* og venter på å låse *mutex1*.

Dette leder til at begge tråder blir hengende i evig tid mens de venter på tilgang som aldri vil komme, siden de låser hverandre.

Videre siden det ikke er implementert noen løsning i *main* som gjør at den venter på at trådene skal bli ferdig, så starter den rett på den evige itereringen av dotter.

Oppgave 3

Programmet nedenfor leser først antall tråder som skal opprettes, n, fra bruker. Deretter startes n tråder som alle kjører funksjonen terningkast():

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

#define MAX_WAIT 2000000

pthread_mutex_t mutex;

void *terningkast(void *nr)
{
    while(1)
    {
        int terning = rand()%6 + 1;

        pthread_mutex_lock(&mutex);
        printf("Traad %3ld -> %d\n", (long) nr, terning);
        ;
        pthread_mutex_unlock(&mutex);

        usleep(rand() % MAX_WAIT + 1);
    }
}

void main()
{
    int n;
    long i;
    pthread_t t;

    printf("n? ");
    scanf("%d", &n);

    pthread_mutex_init(&mutex, NULL);

    for (i = 0; i < n; i++)
        pthread_create(&t, NULL, terningkast, (void *) i);
    ;

    pthread_join(t, NULL);
}
```

Hver av de n trådene kjører en evig løkke. I hvert gjennomløp av løkken ”kastes en terning” ved å velge tilfeldig ett av tallene 1, 2, 3, 4, 5 eller 6. ”Terningkastet” skrives deretter ut og tråden ”sover” en liten stud. Merk at utskriften beskyttes med en mutex, slik at trådene ikke overskriver hverandres output.

Når du kjører dette programmet får du en utskrift som f.eks. kan se slik ut med n=5 tråder:

```
n? 5
Traad    0 -> 2
Traad    1 -> 4
Traad    2 -> 6
Traad    3 -> 5
Traad    4 -> 4
Traad    2 -> 3
Traad    1 -> 3
Traad    2 -> 6
Traad    0 -> 1
Traad    1 -> 5
Traad    0 -> 6
Traad    4 -> 4
Traad    3 -> 3
Traad    2 -> 3
Traad    4 -> 2
Traad    1 -> 6
Traad    3 -> 1
Traad    0 -> 6
Traad    4 -> 2
Traad    2 -> 2 ...
```

a) Bruk en betinget variabel, og skriv om programmet ovenfor slik at:

- Hver gang en tråd får terningkast 1, skal tråden blokkere på den betingede variabelen.
- Hver gang en tråd får terningkast 6, skal tråden sende et signal til den betingede variabelen for å evt. å vekke opp en annen tråd som er blokkert.
- Når en tråd får terningkast 1 eller 6, skal den også skrive ut hvor mange tråder som nå er blokkert.

Utskriften fra programmet ditt kan f.eks. se slik ut med n=5 tråder:

```
n? 5
Traad    0 -> 2
Traad    1 -> 5
Traad    2 -> 6 (0)
```



```
Traad 3 -> 2
Traad 4 -> 4
Traad 1 -> 3
Traad 1 -> 3
Traad 4 -> 6 (0)
Traad 1 -> 1 (1)
Traad 0 -> 1 (2)
Traad 3 -> 5
Traad 2 -> 6 (1)
Traad 1 -> 4
Traad 1 -> 3
Traad 4 -> 2
Traad 3 -> 2
Traad 2 -> 1 (2)
Traad 1 -> 1 (3)
Traad 4 -> 1 (4)
Traad 3 -> 6 (3)
Traad 3 -> 5
Traad 0 -> 5
Traad 3 -> 4
Traad 0 -> 6 (2)
Traad 0 -> 3
Traad 3 -> 6 (1) ...
```

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

#define MAX_WAIT 2000000

pthread_mutex_t mutex;
pthread_cond_t cond;

// Legger paa en global variable som skal brukes til
// aa telle
// antall blokkerte traader
int block_count = 0;

void *terningkast(void *nr)
{
    while(1)
    {
        int terning = rand()%6 + 1;
```

```

        pthread_mutex_lock(&mutex);
// Gjorde om funksjoens struktur til en if test
        if(terning == 1) {
            // Her adderes en verdi til block_count
            // for hver traad
            // som faar terningkast 1.
            block_count++;
            // Traad printer for den stopper, slik at
            // man kan se naar alle
            // traader er blitt blokkert.
            printf("Traad %3ld -> %d (%d)\n", (long)
                nr, terning, block_count);
            // Traad blokkeres aa venter paa et start
            // signal.
            pthread_cond_wait(&cond, &mutex);
            pthread_mutex_unlock(&mutex);
            // Naar traad mottar signal trekker den i
            // fra en verdi paa block_count
            block_count--;
        } else if (terning == 6) {
            // Traader med terningkast 6 sender signal
            // om aa starte en traad i blokk ko
            pthread_cond_signal(&cond);
            // Printer ut med et antall traader i den
            // naavaerende koen
            printf("Traad %3ld -> %d (%d)\n", (long)
                nr, terning, block_count);
            pthread_mutex_unlock(&mutex);
        } else {
            // Den originale handlingen som kjorer for
            // terningkast mellom 2-5
            printf("Traad %3ld -> %d\n", (long) nr,
                terning);
            pthread_mutex_unlock(&mutex);
        }

        usleep(rand() % MAX_WAIT + 1);
    }
}

void main()
{
    int n;
    long i;
    pthread_t t;

```

```

printf("n? ");
scanf("%d", &n);

pthread_mutex_init(&mutex, NULL);
// Initierer betinget blokkering
pthread_cond_init(&cond, NULL);

for (i = 0; i < n; i++)
    pthread_create(&t, NULL, terningkast, (void *) i
    );

pthread_join(t, NULL);
}

```

Listing 2: oppgave_3.c

b) Kjør først programmet ditt med et lite antall tråder, f.eks. $n=5$. Du vil da se at programmet stopper etter en kort stund. Gi deretter programmet et stort antall tråder, f.eks. $n=50$, og la det kjøre lenge. Prøv å gi en forklaring på hva som skjer.

Svar:

Det som skjer her etter en gitt stund er at det til slutt ender opp i en *deadlock* situasjon. For hvert tilfelle hvor det oppstår en betinget blokkering, så vil det åpnes for at en ny tråd kan bruke funksjonen. Som gjør at vi vil med tiden få en samling av tråder som venter på at en annen tråd skal få terningkast 6, slik at en av trådene i køen kan få gå videre.

Dette vil si at den betingende køen vil i gjennomsnitt øke raskere enn antallet tråder som får fjernet blokkeringene. Uavhengig av hvor mange tråder som blir kjørt i dette programmet, så vil til slutt alle tråder ligge i en betinget kø som leder til en *deadlock*. Jo høyere antall tråder som kjører i prosessen, jo lengre tid vil det potensielt ta før programmet går i stå. Men etter en gitt tid vil det stoppe opp uansett.